## STEP 1 — Initialize ChromaDB

```
!pip install chromadb
import chromadb
client = chromadb.Client()
collection = client.create_collection(name="rag_collection")
```

```
Collecting chromadb
  Downloading chromadb-1.5.5-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (7.2 kB)
Collecting build>=1.0.3 (from chromadb)
  Downloading build-1.4.2-py3-none-any.whl.metadata (5.8 kB)
Requirement already satisfied: pydantic>=2.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (2.12.3)
Requirement already satisfied: pydantic-settings>=2.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (2.13.1)
Collecting pybase64>=1.4.1 (from chromadb)
  Downloading pybase64-1.4.3-cp312-cp312-manylinux1_x86_64.manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_5_x86_64.whl
Requirement already satisfied: uvicorn>=0.18.3 in /usr/local/lib/python3.12/dist-packages (from uvicorn[standard]>=0.18.3->chro
Requirement already satisfied: numpy>=1.22.5 in /usr/local/lib/python3.12/dist-packages (from chromadb) (2.0.2)
Requirement already satisfied: typing-extensions>=4.5.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (4.15.0)
Collecting onnxruntime>=1.14.1 (from chromadb)
  Downloading onnxruntime-1.24.4-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (5.2 kB)
Requirement already satisfied: opentelemetry-api>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (1.38.0)
Collecting opentelemetry-exporter-otlp-proto-grpc>=1.2.0 (from chromadb)
  Downloading opentelemetry_exporter_otlp_proto_grpc-1.40.0-py3-none-any.whl.metadata (2.6 kB)
Requirement already satisfied: opentelemetry-sdk>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (1.38.0)
Requirement already satisfied: tokenizers>=0.13.2 in /usr/local/lib/python3.12/dist-packages (from chromadb) (0.22.2)
Collecting pypika>=0.48.9 (from chromadb)
  Downloading pypika-0.51.1-py2.py3-none-any.whl.metadata (51 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 52.0/52.0 kB 2.5 MB/s eta 0:00:00
Requirement already satisfied: tqdm>=4.65.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (4.67.3)
Requirement already satisfied: overrides>=7.3.1 in /usr/local/lib/python3.12/dist-packages (from chromadb) (7.7.0)
Requirement already satisfied: importlib-resources in /usr/local/lib/python3.12/dist-packages (from chromadb) (6.5.2)
Requirement already satisfied: grpcio>=1.58.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (1.78.0)
Collecting bcrypt>=4.0.1 (from chromadb)
  Downloading bcrypt-5.0.0-cp39-abi3-manylinux_2_34_x86_64.whl.metadata (10 kB)
Requirement already satisfied: typer>=0.9.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (0.24.1)
Collecting kubernetes>=28.1.0 (from chromadb)
  Downloading kubernetes-35.0.0-py2.py3-none-any.whl.metadata (1.7 kB)
Requirement already satisfied: tenacity>=8.2.3 in /usr/local/lib/python3.12/dist-packages (from chromadb) (9.1.4)
Requirement already satisfied: pyyaml>=6.0.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (6.0.3)
Requirement already satisfied: mmh3>=4.0.1 in /usr/local/lib/python3.12/dist-packages (from chromadb) (5.2.1)
Requirement already satisfied: orjson>=3.9.12 in /usr/local/lib/python3.12/dist-packages (from chromadb) (3.11.7)
Requirement already satisfied: httpx>=0.27.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (0.28.1)
Requirement already satisfied: rich>=10.11.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (13.9.4)
Requirement already satisfied: jsonschema>=4.19.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (4.26.0)
Requirement already satisfied: packaging>=24.0 in /usr/local/lib/python3.12/dist-packages (from build>=1.0.3->chromadb) (26.0)
Collecting pyproject_hooks (from build>=1.0.3->chromadb)
  Downloading pyproject_hooks-1.2.0-py3-none-any.whl.metadata (1.3 kB)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx>=0.27.0->chromadb) (4.12.1)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx>=0.27.0->chromadb) (2026.2.25)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx>=0.27.0->chromadb) (1.0.9)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx>=0.27.0->chromadb) (3.11)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx>=0.27.0->chroma
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=4.19.0->chromadb) (2
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.12/dist-packages (from jsonschema
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=4.19.0->chroma
Requirement already satisfied: rpds-py>=0.25.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=4.19.0->chromadb)
Requirement already satisfied: six>=1.9.0 in /usr/local/lib/python3.12/dist-packages (from kubernetes>=28.1.0->chromadb) (1.17
Requirement already satisfied: python-dateutil>=2.5.3 in /usr/local/lib/python3.12/dist-packages (from kubernetes>=28.1.0->chro
Requirement already satisfied: websocket-client!=0.40.0,!=0.41.*,!=0.42.*,>=0.32.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kubernetes>=28.1.0->chromadb) (2.32.4
Requirement already satisfied: requests-oauthlib in /usr/local/lib/python3.12/dist-packages (from kubernetes>=28.1.0->chromadb
Requirement already satisfied: urllib3!=2.6.0,>=1.24.2 in /usr/local/lib/python3.12/dist-packages (from kubernetes>=28.1.0->ch
Collecting durationpy>=0.7 (from kubernetes>=28.1.0->chromadb)
  Downloading durationpy-0.10-py3-none-any.whl.metadata (340 bytes)
```

## STEP 2 — Load Embedding Model

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2')
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
modules.json: 100%  349/349 [00:00<00:00, 28.8kB/s]
config_sentence_transformers.json: 100%  116/116 [00:00<00:00, 9.34kB/s]
README.md: 10.5k/? [00:00<00:00, 600kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
sentence_bert_config.json: 100%  53.0/53.0 [00:00<00:00, 6.02kB/s]
config.json: 100%  612/612 [00:00<00:00, 51.5kB/s]
model.safetensors: 100%  90.9M/90.9M [00:01<00:00, 455MB/s]
Loading weights: 100%  103/103 [00:00<00:00, 397.95it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
tokenizer_config.json: 100%  350/350 [00:00<00:00, 13.8kB/s]
vocab.txt: 232k/? [00:00<00:00, 7.38MB/s]
tokenizer.json: 466k/? [00:00<00:00, 19.6MB/s]
special_tokens_map.json: 100%  112/112 [00:00<00:00, 6.40kB/s]
config.json: 100%  190/190 [00:00<00:00, 14.6kB/s]
```

## STEP 3 — Sample Documents

```
documents = [
    "Artificial Intelligence is the simulation of human intelligence.",
    "Machine Learning is a subset of AI.",
    "Deep Learning uses neural networks.",
    "Natural Language Processing deals with text data.",
    "RAG combines retrieval and generation."
]

ids = ["doc1", "doc2", "doc3", "doc4", "doc5"]
```

## STEP 4 — Convert to Embeddings

```
embeddings = model.encode(documents)
```

## STEP 5 — Store in ChromaDB

```
collection.add(
    documents=documents,
    embeddings=embeddings,
    ids=ids
)
```

## STEP 6 — Similarity Search

```
query = "What is AI?"
query_embedding = model.encode([query])
```

```
results = collection.query(
    query_embeddings=query_embedding,
    n_results=2
)

print(results)
```

```
{'ids': [['doc1', 'doc2']], 'embeddings': None, 'documents': [['Artificial Intelligence is the simulation of human intelligence.
```

## STEP 7 — Display Results

```
for doc in results['documents'][0]:
    print(doc)
```

```
Artificial Intelligence is the simulation of human intelligence.
Machine Learning is a subset of AI.
```

## STEP 8 — Retrieval Function

```
def retrieve(query):
    query_embedding = model.encode([query])
    results = collection.query(query_embeddings=query_embedding, n_results=2)
    return results['documents'][0]

print(retrieve("Explain machine learning"))
```

```
['Machine Learning is a subset of AI.', 'Deep Learning uses neural networks.']
```

## STEP 9 — Understanding RAG Workflow

Here's a breakdown of the Retrieval Augmented Generation (RAG) workflow:

- **Step 1: User query** - The process begins with a natural language query from the user.
- **Step 2: Convert query to embedding** - The user's query is converted into a numerical vector (embedding) using an embedding model. This allows for semantic comparison with stored documents.
- **Step 3: Retrieve similar documents** - The query embedding is used to search a vector database (like ChromaDB) to find documents whose embeddings are most similar to the query's embedding. These are the most relevant documents.
- **Step 4: Pass to LLM (next lab)** - The retrieved relevant documents, along with the original user query, are then passed to a Large Language Model (LLM) as context to generate a more informed and accurate response.